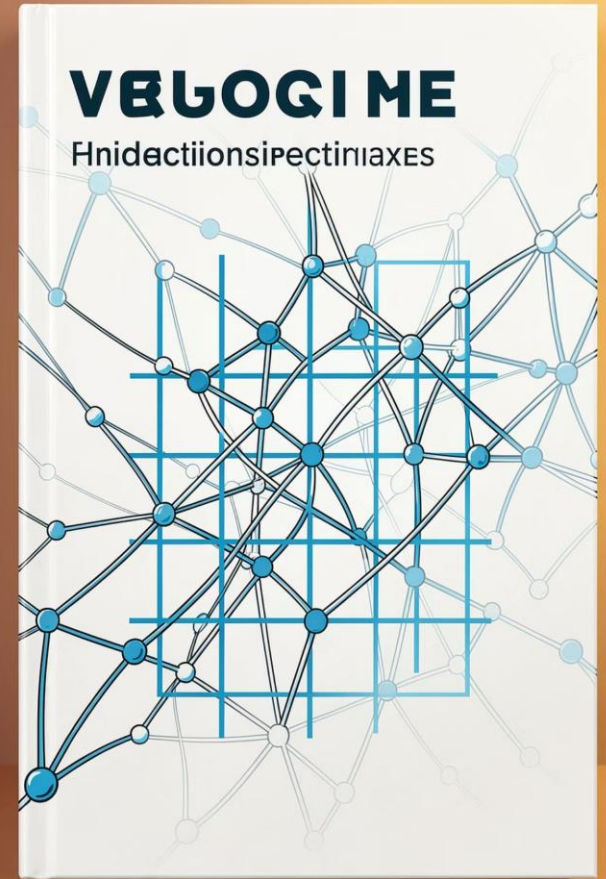


Tic-Tac-Toe AI: Minimax, Alpha-Beta Pruning & Heuristics

CSE 440 — Artificial Intelligence | Section 02 | Group 01

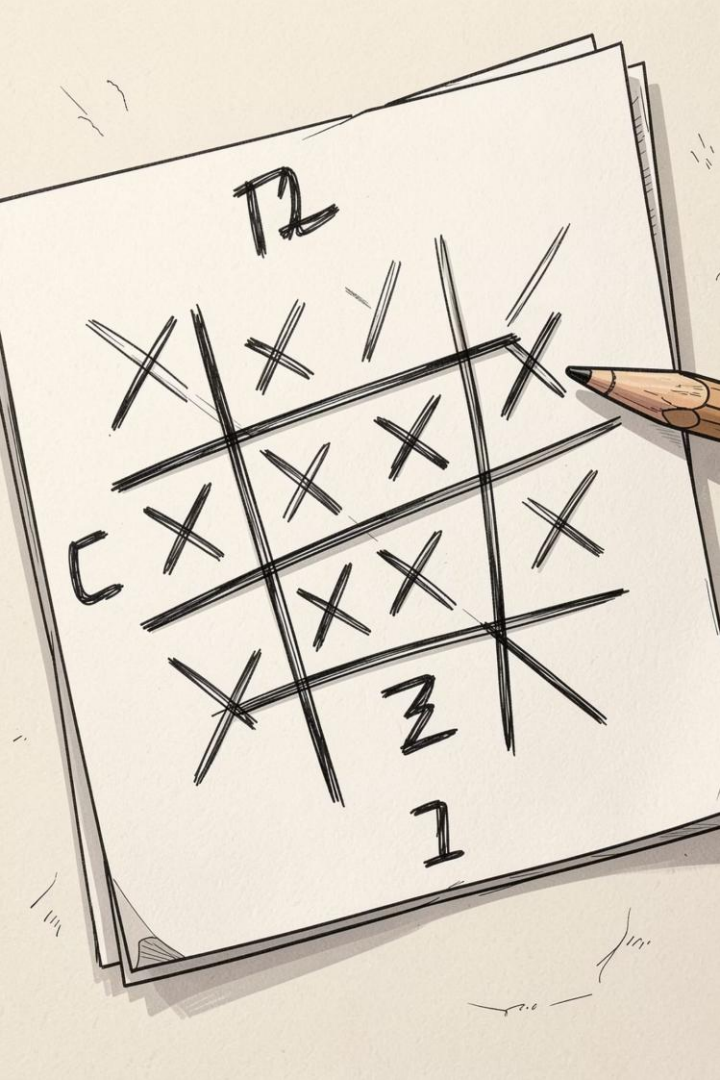
North South University | Summer 2026



Group Members

Name	Student ID
Sourav Roy	2121856042
Sudipta Karmakar	2132245042
Khatune Jannat	2131916642
Tabassum Tasnim Mridula	2211451042

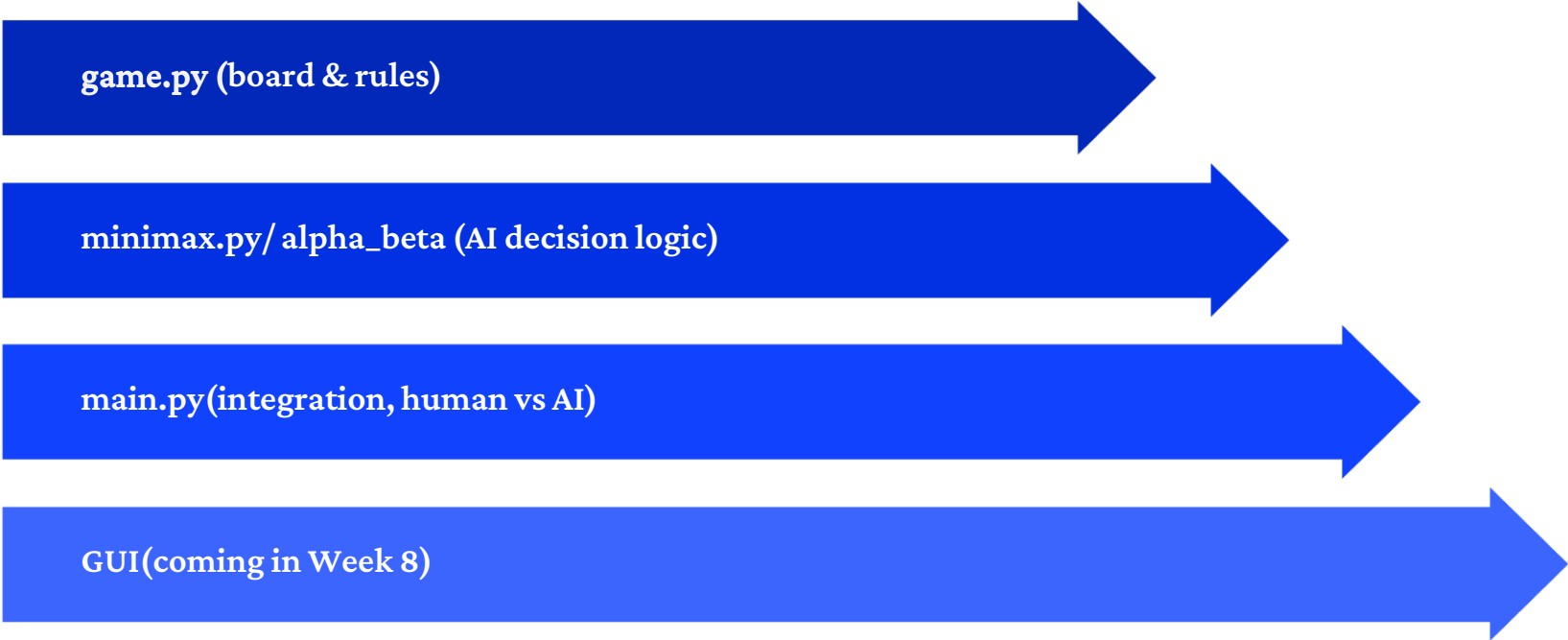
Contact: github.com/souravnewbee/cse440-tictactoe-ai-group01



Project Overview & Objectives

- Build an AI agent that plays Tic-Tac-Toe optimally against a human player.
- Implement Minimax for exhaustive game-tree search and correctness.
- Optimize decision-making with Alpha-Beta pruning to reduce explored nodes.
- Design heuristic evaluation functions for faster play and imperfect-information testing.
- Deliver a playable Human vs AI interface suitable for live demo.

Project Workflow



`game.py` (board & rules)

`minimax.py/ alpha_beta` (AI decision logic)

`main.py`(integration, human vs AI)

GUI(coming in Week 8)

Each module was developed and tested independently, then integrated for full-stack functionality.

```

> ...
import sys
sys.path.append("support")

from game import Game
import minimax
import alpha_beta

def choose_ai():
    print("Choose AI difficulty:")
    print("1. Minimax (slower, always optimal)")
    print("2. Alpha-Beta Pruning (faster, same optimal result)")
    choice = input("Enter 1 or 2: ").strip()
    if choice == "1":
        return minimax
    return alpha_beta

def choose_symbol():
    choice = input("Do you want to be X or O? (X goes first): ").strip()
    if choice not in ("X", "O"):
        print("Invalid choice, defaulting to X")
        return "X"
    return choice

def main():
    print("=== Tic-Tac-Toe: Human vs AI ===\n")
    ai_engine = choose_ai()
    human = choose_symbol()
    ai = "O" if human == "X" else "X"

    game = Game()
    game.print_board()

    while not game.is_game_over():
        if game.current_player == human:
            try:
                pos = int(input(f"Your move ({human}), pick 0-8: "))
            except ValueError:

```

Progress: Weeks 1–3

Week 1 — Repo & Structure

Sourav: repository setup, folder layout, README, contribution guidelines.

Week 2 — Core Game

Sudipta: implemented board representation, CLI interface, win/draw detection and tests.

Week 3 — Minimax

Khatune: implemented Minimax algorithm, validated optimal play on full game tree.

Progress: Weeks 4–5 & Current Status

- Week 4 — Alpha-Beta Pruning optimization (Tabassum): reduced node expansions while preserving optimal moves.
- Week 5 — Full integration & playability (Sourav): main.py ties CLI, AI modules, and tests; playable Human vs AI.
- Status: Fully working — tested on GitHub Codespaces and ready for live demo.

```
---+---+---  
| | O  
---+---+---  
| |
```

Your move (O), pick 0-8: 1

```
X | O | X  
---+---+---  
| | O  
---+---+---  
| |
```

AI (X) is thinking...

AI played position 4

```
X | O | X  
---+---+---  
| X | O  
---+---+---  
| |
```

Your move (O), pick 0-8: 3

```
X | O | X  
---+---+---  
O | X | O  
---+---+---  
| |
```

AI (X) is thinking...

AI played position 6

```
X | O | X  
---+---+---  
O | X | O  
---+---+---  
X | |
```

Roadmap — What's Next



Week 6

Develop and test heuristic evaluation functions for faster approximate play.



Week 7

Performance analysis: benchmark Minimax vs Alpha-Beta vs heuristic-guided search.



Week 8

GUI implementation (Pygame or Tkinter) for user-friendly Human vs AI interaction.



Week 9

Finalize report, prepare final presentation and demo video.

Thank You — Questions?

Repository: github.com/souravnewbee/cse440-tictactoe-ai-group01

Prepared by Group 01 — CSE 440